

Foundations of Artificial Intelligence

M. Helmert
C. Büchner

University of Basel
Spring Term 2026

Exercise Sheet 2 — Solutions

Exercise 2.1 (1+1 marks)

Characterize the following environments by describing if they are *static* / *dynamic*, *deterministic* / *non-deterministic* / *stochastic*, *fully* / *partially* / *not observable*, *discrete* / *continuous*, and *single-agent* / *multi-agent*. Explain your answers.

- (a) Monopoly (https://en.wikipedia.org/wiki/Monopoly_%28game%29)
- (b) Mars rover (https://en.wikipedia.org/wiki/Mars_rover)

Solution:

- (a)
 - static: During your turn, only you can do something.
 - stochastic: The outcome depends on the roll of the dice and which cards are drawn.
 - fully observable: There is no hidden information (if other players want to hide their money you can count the changes).
 - discrete: The money, dice roll etc. are all discrete.
 - multi-agent: It is a multiplayer board game.
- (b)
 - dynamic, non-deterministic, partially observable: We have no control over the environment; for example a meteorite might hit while we're contemplating our next action, and we cannot observe the entire planet and space around it all at once.
 - continuous: The terrain and moving the rover are both continuous (although discretization is possible).
 - single-agent: As far as we know Mars is uninhabited, so there should not be any other agent (assuming we consider a single Mars rover).

Exercise 2.2 (0.5+0.5+0.5+0.5 marks)

Determine if the following statements are true for all state spaces $\mathcal{S} = \langle S, A, cost, T, s_I, S_G \rangle$. Explain your answers.

- (a) If all actions have cost 0, each solution for $\mathcal{S}$ is optimal.
- (b) If $\pi = \langle a_1, \dots, a_n \rangle$ is a minimal cost path from state $s \in S$ to state $s' \in S$ and $\pi' = \langle a'_1, \dots, a'_m \rangle$ is a minimal cost path from s' to state $s'' \in S$, then $\pi'' = \langle a_1, \dots, a_n, a'_1, \dots, a'_m \rangle$ is a minimal cost path from s to s'' .
- (c) It is possible for unit-cost state spaces (where all actions have cost 1) that some state has the same optimal solution cost as one of its successors.
- (d) If there is no solution for some state, then there is no solution for any of its predecessors.

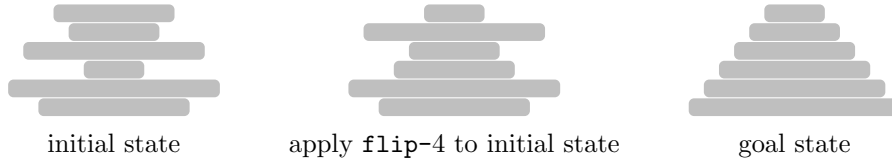
Solution:

- (a) Correct: if all actions have cost 0, any path, and thus also any solution, has cost 0.
- (b) Not correct: there could still be a shorter path from s to s'' that is not via s' (for instance if $s = s''$).

- (c) Correct: consider the state space defined by $S = \{s, s', g\}$, $A = \{a_1, a_2, a_3\}$, $cost(a) = 1$ for all $a \in A$, $T = \{\langle s, a_1, s' \rangle, \langle s, a_2, g \rangle, \langle s', a_3, g \rangle\}$, $s_I = s$, $S_G = \{g\}$. The optimal solution cost for both s and s' is 1.
- (d) Not correct: for example, one of the predecessors could be a goal state.

Exercise 2.3 (2 marks)

In the *pancake* problem we have n pancakes with size $1, \dots, n$ on a pile. Action **flip- i** with cost 1 flips the top i pancakes. The goal is to order the pile by size, i.e. the largest pancake is on the bottom, the second largest on top of the largest and so on. The following figure shows an example instance:



Formalize the state space of the above pancake problem instance. Specify all parts of the state space, i.e. the set of states, the set of actions, the cost function, the set of transitions, the initial state and the set of goal states.

Solution:

We define the state space is $\mathcal{S} = \langle S, A, cost, T, s_I, S_G \rangle$ with

- $S = \{\langle p_0, p_1, p_2, p_3, p_4, p_5 \rangle \mid p_i \in \{1, \dots, 6\}, p_i \neq p_j \text{ for } i \neq j\}$, where p_i denotes the size of the i th pancake from the top.
- $A = \{\text{flip-}i \mid i \in \{1, \dots, 6\}\}$
- $cost(a) = 1$ for all $a \in A$
- $T = \{\langle \langle p_0, p_1, p_2, p_3, p_4, p_5 \rangle, \text{flip-}i, \langle p'_0, p'_1, p'_2, p'_3, p'_4, p'_5 \rangle \rangle \mid \text{flip-}i \in A, p'_j = p_{i-j-1} \text{ if } j < i \text{ and } p'_j = p_j \text{ otherwise}\}$
- $s_I = \langle 3, 2, 5, 1, 6, 4 \rangle$
- $S_G = \{\langle 1, 2, 3, 4, 5, 6 \rangle\}$

This is not the only possible definition. We could for example

- define the states “from bottom to top”, i.e. p_i denotes the i th pancake from the bottom. This will however make the transition definition less intuitive.
- remove action **flip-1**, since applying it to a state s results in s itself.
- not require $p_i \neq p_j$ for $i \neq j$. If an action is applied to a state satisfying this requirement, then the successor will satisfy it as well. Since the initial state satisfies it, all states reachable from s_0 will as well and we thus do not change the reachable state space by dropping the requirement.

Exercise 2.4 (3+1 marks)

The task in this exercise is to write a software program. We expect you to implement your code without using existing code you find online. If you encounter technical problems or have difficulties understanding the task, please let us know.

On the ADAM website of the course, you find Java code (**state-spaces.zip**) that implements the black box interface for state spaces that was discussed in the lecture, as well as an interface for states and actions. The code includes an example implementation of the interface for the blocks world problem. You can test the implementation by invoking the **StateSpaceTest** class, which creates a set of random successor states starting from the initial state.

To run the program, first compile it with

```
javac StateSpaceTest.java
```

from a shell (on Linux) and then run it with

```
java StateSpaceTest blocks blocks-problem.txt
```

The sole purpose of the provided blocks world implementation is to serve as an example that helps you for your own implementation.

Your task is to implement the provided interface for the *pancakes* problem. Everything should be implemented in a single file called `PancakesStateSpace.java`. **In your handin, only submit your `PancakesStateSpace.java` as a solution to this exercise, any other files will be ignored.**

- (a) Implement the state space of the pancakes problem (for a variable number of pancakes).
- (b) Implement the `buildFromCmdline` function for the pancakes problem such that it parses an input file where the pancakes are listed by size from top to bottom, separated by whitespace. The file `pancakes-problem.txt` from the archive encodes the instance from Exercise 2.3. Make sure that your code recognizes invalid inputs such as missing or duplicate pancakes.

You can test your implementation by uncommenting the two indicated lines in the method `createStateSpace` of the `StateSpaceTest` class and then executing the command

```
java StateSpaceTest pancakes pancakes-problem.txt
```

Notice that the command uses the new keyword `pancakes` (rather than `blocks` as in the example above). Also notice that the problem is not expected to be solved by the provided random walk.

Solution:

An example implementation of `PancakesStateSpace.java` is available on ADAM.